

UCS 2312 Data Structures Lab

Assignment 11: Implementation of Searching and Sorting algorithms

Date of Assignment: 11.12.2023

The numberADT contains the size of integer array and an array of integers. [CO2, K3]

- void init(numberADT *N) – To initialize the size of the array
- void insertElements (numberADT *N, int x[10])– To copy the elements from x to the array in numberADT
- void insSort(numberADT *N) – Sorting of elements in the array using selection sort
- void shellSort(numberADT *N) – Sorting of elements in the array using shell sort
- void display(numberADT *N) – Display the elements in the array

CODE:

Arradt.h

```
#include <stdio.h>
#include <stdlib.h>

struct array
{
    int size;
    int arr[100];
};

void init(struct array *a,int s)
{
    a->size=s;
}

void insertElements(struct array *a,int a1[])
{
    a->arr[0]=-999;
    for (int i=0;i<a->size;i++)
    {
        a->arr[i+1]=a1[i];
    }
}

void display(struct array *a)
{
```

```
    for (int i=1;i<=a->size;i++)
    {
        printf("%d ",a->arr[i]);
    }
}

void inssort(struct array *a)
{
    int i,j;
    for (int i=2;i<=a->size;i++)
    {
        j=i;
        while (a->arr[j-1]>a->arr[j])
        {
            int temp=a->arr[j-1];
            a->arr[j-1]=a->arr[j];
            a->arr[j]=temp;
            j=j-1;
        }
    }
}

void shellsort(struct array *a)
{
    int j;
    int incr=a->size/2;
    while (incr>0)
    {
        for (int i=incr+1;i<=a->size;i++)
        {
            j=i-incr;
            while (j>0)
            {
                if (a->arr[j]>a->arr[j+incr])
                {
                    int temp=a->arr[j];
                    a->arr[j]=a->arr[j+incr];
                    a->arr[j+incr]=temp;
                    j=j-incr;
                }
            }
        }
        else
    }
```

```
        j=0;
    }

    }
    incr=incr/2;
}

}

int first(struct array *a,int ele)
{
    int low=0;
    int high=a->size;
    //printf("\n%d ",high);
    int mid;
    while (high>low)
    {
        mid=low+(high-low)/2;
        if (a->arr[mid]==ele)
        {
            if (mid==0 || a->arr[mid-1]!=ele)
                return mid;
            else
                high=mid;
        }
        else if (a->arr[mid]>ele)
            low=mid+1;
        else
            high=mid-1;
    }
}

int last(struct array *a,int ele)
{
    int low=0;
    int high=a->size;
    //printf("\n%d ",high);
    int mid;
    while (high>low)
    {
        mid=low+(high-low)/2;
        if (a->arr[mid]==ele)
```

```
    {
        if (mid==a->size || a->arr[mid+1]!=ele)
            return mid;
        else
            low=mid+1;
    }
    else if (a->arr[mid]>ele)
        low=mid+1;
    else
        high=mid-1;
}
}
```

Arr.c

```
#include <stdio.h>
#include <stdlib.h>
#include "arradt.h"

void main()
{
    struct array *a;
    a=(struct array*)malloc(sizeof(struct array));
    printf("enter the size of the array : ");
    int n;scanf("%d",&n);
    init(a,n);
    int arr[n];
    for (int i=0;i<n;i++)
    {
        printf("enter the element : ");
        int ele;
        scanf("%d",&ele);
        arr[i]=ele;
    }
    insertElements(a,arr);
    printf("\nDISPLAY\n");
    display(a);
    printf("\n\nINSERTION SORT\n");
    inssort(a);
    display(a);
    insertElements(a,arr);
}
```

```
printf("\n\nSHELL SORT\n");
printf("\n");
shellsort(a);
display(a);
printf("\nElement that appears Once\n");
printf("enter the element : ");
int elt;scanf("%d",&elt);
int s1=first(a,elt);
int e1=last(a,elt);
if (s1==e1)
    printf("YES\n");
else
    printf("NO\n");
printf("\nFIND OCCURANCES\n");
printf("enter the element :");
int f;scanf("%d",&f);
int s=first(a,f);

int e=last(a,f);
printf("NUMBER OF OCCURANCES : %d",(e-s+1));
}
```

OUTPUT:

```
enter the size of the array : 5
enter the element : 90
enter the element : 32
enter the element : 21
enter the element : 5
enter the element : 10
```

```
DISPLAY
90 32 21 5 10
```

```
INSERTION SORT
5 10 21 32 90
```

```
SHELL SORT
5 10 21 32 90
```

Additional Routines

Write a routine in ADT to find the element that appears once in sorted array with $O(\log n)$ time complexity.

Write a routine in ADT to count the number of 1's in sorted array with $O(\log n)$ time complexity

ALGORITHM:

Function name:first

Input:

i)struct array *

ii)int

Output:int

1. int low=0;
2. int high=a->size;
3. int mid;
4. while (high>low)
{
 mid=low+(high-low)/2;
 if (a->arr[mid]==ele)
 {
 if (mid==0 || a->arr[mid-1]!=ele)
 return mid;
 else
 high=mid;
 }
5. else if (a->arr[mid]>ele)
 low=mid+1;
6. else
 high=mid-1;
}

Function name:last

Input:

i)struct array *

ii)int

Output:int

1. int low=0;
2. int high=a->size;
3. int mid;
4. while (high>low)
{
 mid=low+(high-low)/2;

CSE B

```
        if (a->arr[mid]==ele)
        {
            if (mid==a->size || a->arr[mid+1]!=ele)
                return mid;
            else
                low=mid+1;
        }
5. else if (a->arr[mid]>ele)
    low=mid+1;
6. else
    high=mid-1;
}
```

CODE:

Arradt.h

```
int first(struct array *a,int ele)
{
    int low=0;
    int high=a->size;
    //printf("\n%d ",high);
    int mid;
    while (high>low)
    {
        mid=low+(high-low)/2;
        if (a->arr[mid]==ele)
        {
            if (mid==0 || a->arr[mid-1]!=ele)
                return mid;
            else
                high=mid;
        }
        else if (a->arr[mid]>ele)
            low=mid+1;
        else
            high=mid-1;
    }
}

int last(struct array *a,int ele)
{
    int low=0;
```

```
int high=a->size;
//printf("\n%d ",high);
int mid;
while (high>low)
{
    mid=low+(high-low)/2;
    if (a->arr[mid]==ele)
    {
        if (mid==a->size || a->arr[mid+1]!=ele)
            return mid;
        else
            low=mid+1;
    }
    else if (a->arr[mid]>ele)
        low=mid+1;
    else
        high=mid-1;
}
}
```

Arr.c

```
Void main()
{
    printf("\nElement that appears Once\n");
    printf("enter the element : ");
    int elt;scanf("%d",&elt);
    int s1=first(a,elt);
    int e1=last(a,elt);
    if (s1==e1)
        printf("YES\n");
    else
        printf("NO\n");
    printf("\nFIND OCCURANCES\n");
    printf("enter the element :");
    int f;scanf("%d",&f);
    int s=first(a,f);

    int e=last(a,f);
    printf("NUMBER OF OCCURANCES : %d",(e-s+1));
}
```

OUTPUT:

```
5 10 21 32 90
Element that appears Once
enter the element : 5
YES

FIND OCCURANCES
enter the element :10
NUMBER OF OCCURANCES : 1
```

LEARNING OUTCOME:

After the completion of this exercise, I've learnt about two sorting techniques
Insertion and shell sorting.

I have learnt to modify binary search to find occurrences of a specific element in $O(\log n)$